Lab2:BombLab 报告

姓名：龚彦哲
学号：26303050100
注：请先从左往右，再从上往下看（很抱歉格式实在丑陋）

一. 每个已完成关卡的关键汇编&恢复出的计算关系&推理过程

1. solve_phase_1

(1) 关键汇编:1)
```
4020d3: cmp    $0x1,%eax
```
count=1

```
402055: mov    %rdi,%rax
402058: mov    %rdi,%rdx
40205b: mov    %edi,%esi
40205d: shr    $0x20,%rax
402061: mov    %edi,%r8d
402064: shr    $0x10,%rdx
402068: mov    %eax,%edi
40206a: mov    %rsp,%rbp
```
2) 将 challenge 右移不同位数给后续运算

```
0000000000402020 <p1_step0>:
402020: endbr64
402024: sub    %esi,%edi
402026: lea    0xfbe(%rdi,%rdi,2),%eax
40202d: ret
```
3)

```
0000000000402030 <p1_step1>:
402030: endbr64
402034: xor    $0xf277,%esi
40203a: lea    (%rsi,%rdi,8),%eax
40203d: ret
```

```
0000000000402040 <p1_step2>:
402040: endbr64
402044: lea    (%rdi,%rsi,4),%eax
402047: xor    $0x4b4b,%eax
40204c: ret
```
主要运算得到结果

(2) 计 算 关 系 ：

```
28    int a16=(uint32_t)(challenge>>16);
29    int High32=(uint32_t)(challenge>>32);
30    int Low32=(uint32_t)(challenge&0xFFFFFFFF);
31    values[0]=(uint32_t)(((Low32*4+High32)^0x4b4b)^0xf277) +(uint32_t)(((Low32-a16)*3+0xfbe)*8);
```

28-30 行对应关键汇编 1)，31 行对应关键汇编 2)

(3)推理过程:1)sscanf 读入 1 个 %u 到 rbp-0x14,与 p1_compute 返回值比较,得"input == 计算值"的判定。

2)shr $0x20 / shr $0x10 配合后续按 32 位使用，确定三个字段为 High32、a16、Low32。
3) lea、xor、sub 直接对应 C 表达式中的乘法、异或、减法，无需额外推断。
4)按调用地址顺序(0x40206d → 0x40207a → 0x402083)确定执行顺序为 step2 → step0 → step1，而非按命名的 0/1/2。

2. solve_phase_2

(1) 关键汇编:1)

```
48    40214e: call   4011e0 <__isoc99_sscanf@plt>
49    402153: pop    %rdx
50    402154: pop    %rcx
51    402155: cmp    $0x6,%eax
```

得出 count=6

2)

```
40215a: mov    %r13,%rdx
40215d: mov    -0x40(%rbp),%eax
402160: shr    $0x28,%rdx
402164: movzbl %dl,%edx
402167: lea    (%rdx,%rdx,2),%edx
40216a: cmp    %edx,%eax
40216c: jne    402115 <phase_02+0x25>
```

得到 values[0]

3)

```
402178: mov    %r13,%rdx
40217b: add    %eax,%eax
40217d: shr    %cl,%rdx
402180: and    $0x1f,%edx
402183: xor    %edx,%eax
402185: lea    (%rax,%rsi,8),%edx
402188: mov    -0x40(%rbp,%rsi,4),%eax
40218c: cmp    %edx,%eax
40218e: jne    402115 <phase_02+0x25>
402190: add    $0x1,%rsi
402194: add    $0x3,%ecx
402197: cmp    $0x6,%rsi
40219b: jne    402178 <phase_02+0x88>
```

循环计算 values[1]到 values[5]

(2) 计算关系:1)

```
Test2=((challenge>>0x28)&0xFF)*3;
values[0]=(uint32_t)Test2;
```

对应关键汇编 2)

```
uint64_t Move2=0xc;
Left2=(uint32_t)Test2;
for(uint32_t i=1;i<=5;i++)
{
    Left2*=2;
    Try2=challenge>>(Move2&0xFF);
    Right2=(uint32_t)(Try2&0x1F);
    Left2=Left2^Right2;
    Right2=8*i+Left2;
    values[i]=Right2;
    Left2=Right2;
    Move2+=3;
}
```

2) 其中 for 循环对应关键汇编 3)

(3)推理过程:1)mov $0x6,%esi & cmp $0x6,%eax：确定本关要 6 个数。

2)mov $0xc,%ecx 初始化、add $0x3,%ecx 步进，对应 C 里 Move2 从 0xc 起、每次 +3。

3)shr$0x28,%rdx&movzbl%dl,%edx&lea(%rdx,%rdx,2) 即((challenge>>40)&0xFF)*3，对应 Test2。

4)循环内 add %eax,%eax（×2）、shr %cl,%rdx（按 Move2 移位）、and $0x1f（取 5 位）、xor %edx,%eax、lea (%rax,%rsi,8)（8*i+Left2），与 C 中循环体逐句一一对应，寄存器 esi 即 i、ecx 即 Move2、eax 即 Left2。

5)cmp $0x6,%rsi（i 与 6 比）+ jne：确定循环条件为 i<=5。

3.solve_phase_3

(1)关键汇编:1) `4022d7: cmp $0x2,%eax` count=2

2)
```
4022e1: and    $0x7,%edi
4022e4: cmp    %edi,-0x18(%rbp)
4022e7: jne    4022b5 <phase_03+0x25>
```
得到 values[0]

```
40223d: notrack jmp *0x405820(,%rax,8)
402245: nopl   (%rax)
402248: jmp    4021c0 <p3_case_1>
40224d: nopl   (%rax)
402250: jmp    402220 <p3_case_7>
402252: nopw   0x0(%rax,%rax,1)
402258: jmp    4021f0 <p3_case_4>
40225a: nopw   0x0(%rax,%rax,1)
402260: jmp    402210 <p3_case_6>
402262: nopw   0x0(%rax,%rax,1)
402268: jmp    4021d0 <p3_case_2>
40226d: nopl   (%rax)
402270: jmp    4021e0 <p3_case_3>
402275: nopl   (%rax)
402278: jmp    402200 <p3_case_5>
40227a: nopw   0x0(%rax,%rax,1)
402280: jmp    4021b0 <p3_case_0>
402285: xor    %eax,%eax
402287: ret
```

3) 通过 switch 语句得出 values[1](具体的 p3_case 省略，其中均为计算语句)

```
uint32_t Test31=(uint32_t)(challenge>>0x20);
uint32_t Test32=(uint32_t)(challenge>>0x3);
uint32_t Test3=Test31^Test32;
uint32_t Last3=(uint32_t)challenge&0x7;
```

(2) 计 算 过 程 :1) `values[0]=Last3;` 对应关键汇编 2)

```
switch(Last3)
{
    case 5:
        ReTest3=(Test3>>1)+(Test3^0xe8d6);
        break;
    case 1:
        ReTest3=(Test3*2)^0xdba6;
        break;
    case 2:
        ReTest3=Test3*5+0x31867;
        break;
    case 3:
        ReTest3=Test3*5+0x8d18;
        break;
    case 0:
        ReTest3=(Test3*2)^0x2729;
        break;
    case 4:
        ReTest3=Test3*5+0xed03;
        break;
    case 6:
        ReTest3=Test3*5+0x62a5;
        break;
    case 7:
        ReTest3=(Test3>>1)+(Test3^0x5b0b);
        break;
}
values[0]=Last3;
values[1]=(uint32_t)ReTest3;
```

2)

对应关键汇编 3)(注意到 case 0,4,5 位置发生轮换，这是多次试错积累的结果，但是具体原因未知……)

(3)推理过程：1)cmp $0x2,%eax：本关要 2 个数，分别存在 rbp-0x18、rbp-0x14。

2)and $0x7,%edi & cmp %edi,-0x18(%rbp)：确定 values[0] == challenge & 0x7，即 Last3 同时是分支选择子。

3)shr $0x3 与 shr $0x20 后 xor：确认 Test3 = (challenge>>3) ^ (challenge>>32)。

4)call p3_dispatch 的返回值与 rbp-0x14 比较、sete %al：确定 values[1] 是分支计算结果。

5)通过 switch 与每个 case 中的公式计算 values[1]。

4.solve_phase_4

(1)关键汇编:1) `4023c3: cmp $0x1,%eax` count=1

```
4023c8: mov    %r12,%rcx
4023cb: mov    %r12,%rax
4023ce: mov    %r12d,%edi
4023d1: xor    %esi,%esi
4023d3: shr    $0x20,%rax
4023d7: shr    $0x7,%rcx
4023db: and    $0x7f,%edi
4023de: mov    $0x7f,%edx
```

2) 4023e3: xor %eax,%ecx p4_walk 函数所需变量的准备工作

```
402318: sub    %esi,%edi
40231a: shr    $1,%edi
40231c: add    %esi,%edi
40231e: cmp    %eax,%edi
402320: je     402370 <p4_walk+0x60>
```

3)

```
40233c: lea    (%rax,%rax,2),%eax
40233f: xor    %ebx,%eax
402327: lea    (%rdi,%rcx,1),%ebx
40232a: sub    $0x8,%rsp
40232e: cmp    %edi,%eax
402330: jb     402350 <p4_walk+0x40>
402332: lea    0x1(%rdi),%esi
402335: mov    %eax,%edi
402337: call   402310 <p4_walk>
40235a: lea    (%rax,%rax,2),%eax
40235d: xor    %ebx,%eax
```

均为 p4_walk 中的关键计算类语句

(2)计算过程：

1)

```
uint32_t r=(uint32_t)(challenge&0x7fULL);
uint32_t x=0;
uint32_t c=0x7f;
uint32_t d=(uint32_t)(((challenge>>32)^(challenge>>7))&0xffffffffULL);
```

对应关键汇编 2）

2)

```
static int p4_walk(uint32_t r, uint32_t x, uint32_t c, uint32_t d) {
    uint32_t m=x+((c-x)>>1);
    if (r==m)
    {
        return (int)((m+0xa5u)^d);
    }

    uint32_t ebx=m+d;

    if (r>m)
    {
        uint32_t rec=(uint32_t)p4_walk(r,m+1,c,d);
        return (int)(((rec*3u)^ebx)^0xau);
    }
    else
    {
        uint32_t rec=(uint32_t)p4_walk(r,x,m-1,d);
        return (int)(((rec*3u)^ebx)^0x13u);
    }
}
```

对应关键汇编 3)

(3)推理过程:1)and $0x7f,%edi → r；mov $0x7f,%edx → c；xor %esi,%esi → x=0；shr $0x20 与 shr $0x7 后 xor → d。四个参数按 rdi=r, rsi=x, rdx=c, rcx=d 传入，与函数签名一致。

2)sub %esi,%edi & shr $1 & add %esi,%edi 即 m = x + ((c-x)>>1)，二分中点。

3)lea (%rdi,%rcx,1),%ebx = m + d（rdi=m, rcx=d），是后续两分支共用的 ebx

4)cmp %edi,%eax & jb 402350：r 小于 m 走 else。两分支尾部的 lea (%rax,%rax,2)+xor %ebx 是 rec*3 ^ ebx，再分别异或 0xa（r>m）或 0x13（r<m），与 C 完全对应。

5.solve_phase_5

(1)关键汇编：1)

```
4024a7: cmp    $0x2,%eax
4024b3: call   402400 <p5_traverse>
4024b8: mov    %eax,%edx
4024ba: xor    %eax,%eax
4024bc: cmp    %edx,-0x1c(%rbp)
4024bf: jne    402485 <phase_05+0x25>
4024c1: mov    -0x14(%rbp),%eax
```

count=2

2) 表示 values[0] 和 values[1]

```
402444: mov    0x8(%rdx),%rdx
402448: add    $0x1,%ecx
40244b: mov    0x4(%rdx),%esi
40244e: xor    (%rdx),%eax
402450: add    %esi,%eax
402452: mov    %esi,(%r8)
402455: cmp    %ecx,%edi
402457: jae    402444 <p5_traverse+0x44>
```

3) 其实<p5_traverse>整体都是计算，这里只截取其中 do-while 循环部分

```
int result = 0;
uint32_t ret = p5_traverse(challenge, &result);

values[1] = (uint32_t)result;
values[0] = ret;
count = 2;
```

(2)计算关系:1) 对应关键汇编

2)

2)

```
static uint32_t p5_traverse(uint64_t challenge, int *result_out)
{
    uint32_t eax = (uint32_t)challenge;
    uint32_t sh3 = (uint32_t)(challenge >> 3);
    uint32_t hi  = (uint32_t)(challenge >> 32);
    uint32_t sh6 = (uint32_t)(challenge >> 6);
    unsigned idx = (uint32_t)challenge & 7u;
    int cur = p5_head_idx[idx];
    int esi = p5_raw[cur].val1;
    eax  = sh6 ^ hi;
    eax ^= (uint32_t)p5_raw[cur].val0;
    eax ^= 0x49fau;
    *result_out = esi;
    eax += (uint32_t)esi;
    uint32_t n = sh3 & 3u;
    if (n == 0)
        return eax;
    uint32_t cnt = 1;
    do {
        cur = p5_raw[cur].next_idx;
        cnt++;
        esi = p5_raw[cur].val1;
        eax ^= (uint32_t)p5_raw[cur].val0;
        eax += (uint32_t)esi;
        *result_out = esi;
    } while (n >= cnt);
    return eax;
}
```

其中 do-while 循环部分对应关键汇编 3)

```
static const struct p5_raw p5_raw[] =
{
    { 0x8d8f, 0x0245,  8 },
    { 0xe1ef, 0x0520,  11 },
    { 0x8d9a, 0x036f,  0 },
    { 0x7097, 0x007c,   10 },
    { 0x64ca, 0x0b67,  7 },
    { 0x8fbd, 0x0a6e,  9 },
    { 0x790d, 0x0815,  3 },
    { 0x768c, 0x0919, -1 },
    { 0x5e1a, 0x0642,  1 },
    { 0x81e3, 0x0112, 2},
    { 0x5484, 0x073d, 4},
    {0xba31 ,0x0471, 6},
};
```

3) 汇编语言中通过地址进行运算，这里将地址转换为下标，并制表以供计算时获取数据

(3)推理过程:1)and $0x7,%eax 取低 3 位作 idx，shr $0x3,%rcx 取 sh3，与 C 代码一致。

2)mov 0x405b60(,%rax,8),%rdx：按 idx*8 索引一个指针表（p5_head_idx），得到起始节点指针 cur，对应 p5_head_idx[idx]。

3)mov 0x4(%rdx),%esi 取节点偏移 +4 的字段 = val1；xor (%rdx),%eax 取偏移 0 的字段 = val0；mov 0x8(%rdx),%rdx 取偏移 +8 字段 = next_idx。由此确定 struct 布局 {val0, val1, next_idx}。

4)shr $0x20 / shr $0x6 后 xor，加 ^= val0、^= 0x49fa、+= val1，与 C 逐句对应。

5)and $0x3,%edi 得 n；je 判 n==0 直接返回；否则 cnt=1 起循环，jae 即 n >= cnt 时继续，即循环体执行 n 次（节点数 n+1）。

6)主函数 cmp $0x2,%eax 确定读 2 值；随后 input[0] 对 ret、input[1] 对 result 分两次判等，故 values[0]=ret、values[1]=result。

6.solve_phase_6

(1):关键汇编 1) `4025f3: cmp $0x1,%eax` count=1

```
0000000000402560 <p6_dispatch>:
  402560: endbr64
  402564: push   %rbp
  402565: mov    %rsp,%rbp
  402568: push   %rbx
  402569: mov    %rdi,%rbx
  40256c: shr    $0x6,%rdi
  402570: mov    %rbx,%rax
  402573: shr    $0x20,%rax
  402577: xor    %eax,%edi
  402579: mov    %rbx,%rax
  40257c: sub    $0x8,%rsp
  402580: and    $0x38,%ebx
  402583: and    $0x7,%eax
  402586: call   *0x405e00(,%rax,8)
  40258d: xor    $0xbe40,%eax
  402592: mov    %eax,%edi
  402594: call   *0x405e00(%rbx)
  40259a: mov    -0x8(%rbp),%rbx
  40259e: leave
  40259f: add    $0xa9e1,%eax
  4025a4: ret
```

2) 先计算得到%rax，进而通过 2 次连续 call 先得到%rbx 再得到 values[0]（call 出的算式内容不在汇编语言中，我通过 dump 得到对应地址再访问对应地址以得到隐含的公式，具体见计算过程）

```
405e00 30254000 00000000 10254000 00000000
405e10 f0244000 00000000 e0244000 00000000
405e20 d0244000 00000000 40254000 00000000
405e30 d0244000 00000000 40254000 00000000
```

```
uint32_t t1   = (uint32_t)(arg >> 6) ^ (uint32_t)(arg >> 32);
uint32_t idx1 = (uint32_t)(arg & 0x7);
uint32_t idx2 = (uint32_t)((arg >> 3) & 0x7);
uint32_t r1   = p6_table[idx1](t1);
r1 ^= 0xbe40;
uint32_t r2   = p6_table[idx2](r1);
return r2 + 0xa9e1;
```

(2)计算过程：1) 对应关键汇编 2)

2)

```
static uint32_t p6_op_0(uint32_t x) { return (x + 0x4d1e) ^ (x >> 1); }
static uint32_t p6_op_1(uint32_t x) { return ((x << 5) | (x >> 27)) ^ 0xc795; }
static uint32_t p6_op_2(uint32_t x) { return (x * 9 + 0x6e8a) ^ (x * 3); }
static uint32_t p6_op_3(uint32_t x) { return (x ^ (x >> 7)) + 0x43cb; }
static uint32_t p6_op_4(uint32_t x) { return (x >> 2) | (x << 30); }
static uint32_t p6_op_5(uint32_t x) { return (x ^ 0x62ac) + (x << 5);

static uint32_t (*p6_table[8])(uint32_t) = {
    p6_op_4,  /* idx=0 */
    p6_op_3,  /* idx=1 */
    p6_op_2,  /* idx=2 */
    p6_op_1,  /* idx=3 */
    p6_op_0,  /* idx=4 */
    p6_op_5,  /* idx=5 */
    p6_op_0,  /* idx=6 */
    p6_op_5,  /* idx=7 */
};
```

p6_op 对应关键汇编 2)中附加图片中的地址中隐含的公式,p6_table 对应附加图片中地址与公式的对应关系，由 8 个地址与初地址相减除以 16 得到

(3)推理过程:1)cmp $0x1,%eax：本关只校验 1 个字段。

2)shr $0x6 与 shr $0x20 后 xor：确定 t1 = (challenge>>6) ^ (challenge>>32)。

3)and $0x7,%eax 得 idx1；and $0x38,%ebx 得到的是 idx2*8，作 call *0x405e00(%rbx) 的表偏移，故 idx2 = (challenge>>3) & 7。两次 call *(table + k*8) 确认是通过函数指针表做 2 次分派。

4)两次调用之间 xor $0xbe40,%eax，且 mov %eax,%edi 把结果作为第二次调用的入参，确定 r1 混合后喂给 idx2 对应算子。

5)结尾 add $0xa9e1,%eax 即最终 +0xa9e1。

6)表项与算子函数的对应关系由 C 注释中的 0x405e00 的 8 项简化给定,故 idx 到 p6_op_k 的映射直接采用题给顺序，哪些 idx 复用同一算子（idx4/6 用 op_0，idx5/7 用 op_5）也据此确定。

7.solve_secret

(1)关键汇编:1) `40268b: cmp $0x3,%eax` 输出%u 有 3 个

```
402770: mov    %rbx,%rsi
402773: mov    %r12,%rdi
402776: call   4026f0 <secret_copy>
40277b: cmpl   $0x4e45504f,-0xb0(%rbp)
402785: jne    40275b <secret_gate+0x4b>
```

2) 确保输出时 OPEN 在指定位置

```
0000000000402610 <secret_mix>:
  402610: endbr64
  402614: mov    %edi,%eax
  402616: xor    $0xad7d,%edi
  40261c: shr    $0x7,%eax
  40261f: xor    $0xf3bb,%eax
  402624: add    %edi,%eax
  402626: ret
```

3) 计算相关的汇编语言（其实 secret_copy 与 secret_phase 中也有大量运算，且 secret_mix 被 secret_phase 调用，不过这里就不完全列出了）

(2)计算过程:

```
int n = snprintf(out, cap, "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAOPEN %u %u %u", ecx, eax, third);
return (n >= 0 && (size_t)n < cap) ? 0 : 1;
```

1) 对应关键汇编 2),补充了 32 个 A 作为填充

```
uint32_t secret_mix(uint32_t x) {
    return ((x >> 7) ^ 0xf3bb) + (x ^ 0xad7d);
}
```

2) 对应关键汇编 3)，以及

```
uint32_t ecx = (uint32_t)(context & 3);
uint32_t r9d = ecx * 0xad7d;
r9d ^= (uint32_t)(context >> 16);

uint32_t esi = (uint32_t)((challenge >> 32) ^ (uint32_t)challenge);
uint32_t r8d = (uint32_t)((context >> 32) ^ (uint32_t)context);
uint32_t edi = esi ^ r8d;

uint32_t eax   = secret_mix(edi) ^ r9d;
uint32_t edx   = (uint32_t)(0xf3bb + r8d + ecx) + (esi >> 7);
uint32_t third = eax ^ edx;
```

对应 secret_copy 与 secret_phase 中的大量运算

(3) 推理过程:1)secret_gate 长度判断:sub $0x24,%rax（0x24=36）+cmp $0x7f（127)+jbe：strlen(input)-36<=127，即长度落在 [36, 163] 才继续。

2)OPEN 标记：cmpl $0x4e45504f,-0xb0(%rbp)，0x4e45504f 小端 = 'O''P''E''N'，即校验输入中含 OPEN；与 snprintf 里 "...OPEN %u %u %u" 呼应。

3)secret_copy 与 secret_phase 中汇编语言与 C 语言代码相对应（此处省略）

4)secret_mix 逐条还原 xor $0xad7d → x^0xad7d；shr $0x7 + xor $0xf3bb

→ (x>>7)^0xf3bb；add → 相加，即 ((x>>7)^0xf3bb) + (x^0xad7d)。

5)最终输出由 solve_secret 的 snprintf 确定：前缀 "A...AOPEN " + 3 个 %u，即 exa、第 1 项、第 2 项。

二.每个已完成关卡至少一次使用公开列表之外的 seed 验证的记录

注：我选择使用 seed=2(公开列表之外)，方法是将 defuser.c 中的程序的 return 系统改为 scanf 与 printf 系统之后新建一个.c 代码，在新代码中输入 seed2 对应的 challenge 就可以得到在运行炸弹时的正确输入，附图如下：

```
l1vac@LAPTOP-V0S5QA84:/mnt/c/Users/L1vAc/Desktop/LabZip/L1vAcLab2-main/L1vAcLab2-main$ ./bomb --seed 2 --phase 1
BOMBLAB 2026                                    这里还没有明天

                    *
                     \
                  .-"""-.
                .'       '.
               /    LOOP    \
              |               |
              \               /
               '-----------'

[1 当前] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xbfc846100bfc1e42
Input (unsigned decimal integers):
2111453860
Phase 1 defused.
```

```
l1vac@LAPTOP-V0S5QA84:/mnt/c/Users/L1vAc/Desktop/LabZip/L1vAcLab2-main/L1vAcLab2-main$ ./bomb --seed 2 --phase 2
BOMBLAB 2026                                    这里还没有明天

                    *
                     \
                  .-"""-.
                .'       '.
               /    LOOP    \
              |               |
              \               /
               '-----------'

[1 待解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]

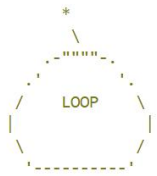
单关调试：只记录本次通过的关卡，不播放完整结局。

第 2 关 · 钟楼
钟楼没有钟声。齿轮照常转动，日期却停在昨天。
林夏翻开底座里的记录：同一天的几次校时被写在不同位置，
每一次都接着上一次的结果。你们得先确认这串记录能接起来。
窗外的送货车又绕过街角。司机还是刚才那句话：
‘今天来不及了。明天一定送到。’

Phase 2 challenge: 0x987bbcbfdd7e532f
Input (unsigned decimal integers):
564 1141 2310 4651 9341 18703
Phase 2 defused.
```

```
l1vac@LAPTOP-V0S5QA84:/mnt/c/Users/L1vAc/Desktop/LabZip/L1vAcLab2-main/L1vAcLab2-main$ ./bomb --seed 2 --phase 3
```

BOMBLAB 2026　　这里还没有明天



[1 待解除] [2 待解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

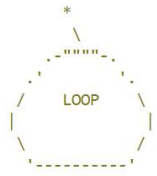
单关调试：只记录本次通过的关卡，不播放完整结局。

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

```
Phase 3 challenge: 0xc3f2827affe7f664
Input (unsigned decimal integers):
4 206134417
Phase 3 defused.
```

```
l1vac@LAPTOP-V0S5QA84:/mnt/c/Users/L1vAc/Desktop/LabZip/L1vAcLab2-main/L1vAcLab2-main$ ./bomb --seed 2 --phase 4
```

BOMBLAB 2026　　这里还没有明天



[1 待解除] [2 待解除] [3 待解除] [4 当前] [5 待解除] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 4 关 · 档案馆
档案馆里没有人，书架上的指示灯一层层向深处延伸。
原来的存档经过几次搬迁，被分在不同的目录里。
你们找到入口，后面的路还要沿着旧索引逐层查下去。
林夏把刚才的纸翻到背面：‘这次别凭房间号猜，跟着记录走。’

```
Phase 4 challenge: 0x4fc446b53f17fb29
Input (unsigned decimal integers):
1324015274
Phase 4 defused.
```

```
l1vac@LAPTOP-V0S5QA84:/mnt/c/Users/L1vAc/Desktop/LabZip/L1vAcLab2-main/L1vAcLab2-main$ ./bomb --seed 2 --phase 5
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 当前] [6 待解除]

单关调试：只记录本次通过的关卡，不播放完整结局。

第 5 关 · 巡检回廊
巡检回廊连着档案馆和街区。墙上的设备按编号排得很整齐，
线缆却从一台背后穿过去，接到另一排的某个接口。
维修铺的老板把一盏没装好的灯放到窗边。
底座上贴着纸条：‘今天做好接口，明天装上。’
林夏认出那盏灯：‘我们刚进来时，他就在做这个。’

Phase 5 challenge: 0x58bc3cb37bc7b2b3
Input (unsigned decimal integers):
2505269890 1312
Phase 5 defused.
```

```
l1vac@LAPTOP-V0S5QA84:/mnt/c/Users/L1vAc/Desktop/LabZip/L1vAcLab2-main/L1vAcLab2-main$ ./bomb --seed 2 --phase 6
BOMBLAB 2026                                    这里还没有明天

                        *
                         \
                      .-"""-.
                    .'       '.
                   /    LOOP    \
                  |               |
                   \             /
                    '-----------'

[1 待解除] [2 待解除] [3 待解除] [4 待解除] [5 待解除] [6 当前]

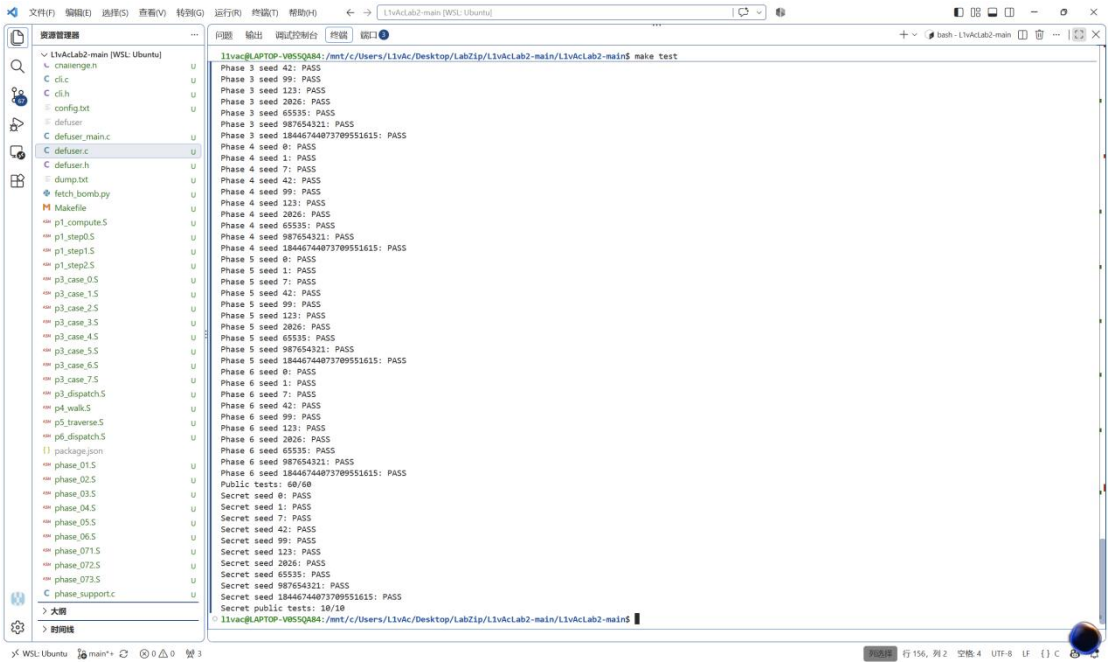
单关调试：只记录本次通过的关卡，不播放完整结局。

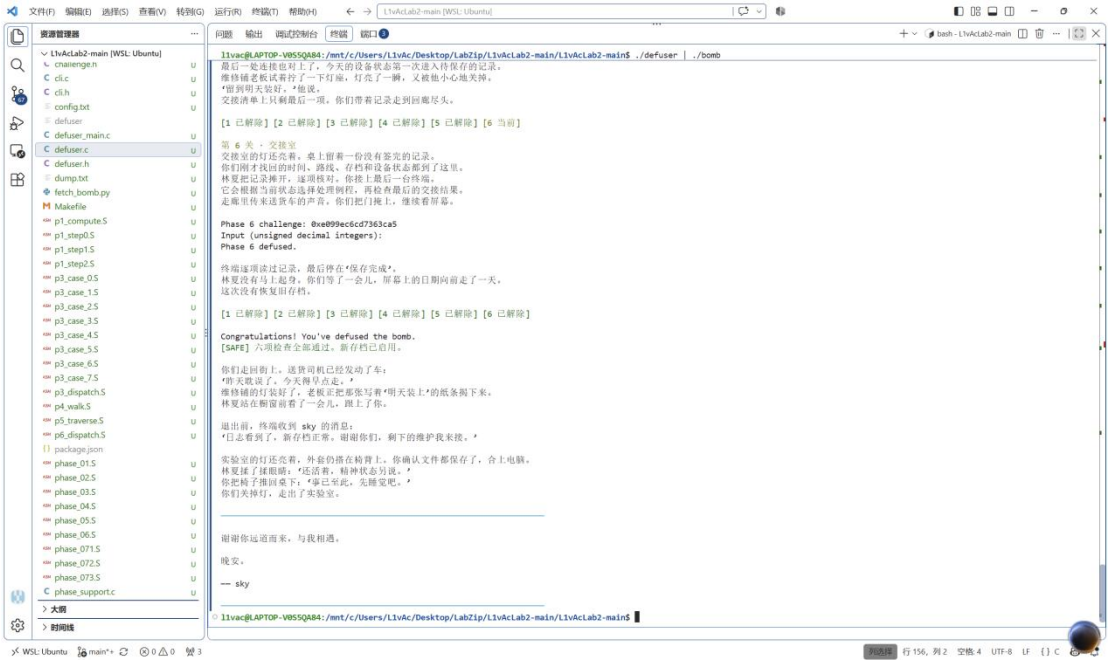
第 6 关 · 交接室
交接室的灯还亮着。桌上留着一份没有签完的记录。
你们刚才找回的时间、路线、存档和设备状态都到了这里。
林夏把记录摊开，逐项核对。你接上最后一台终端。
它会根据当前状态选择处理例程，再检查最后的交接结果。
走廊里传来送货车的声音。你们把门掩上，继续看屏幕。

Phase 6 challenge: 0xb9f24f7bae4a6586
Input (unsigned decimal integers):
1597790256
Phase 6 defused.
```

三. 已完成关卡的运行结果截图

注：以下 2 张图片分别是运行./defuser | ./bomb 和运行 make test 的结果，均通过





四. 引用资料

本次实验我几乎没有使用 gdb，而是通过离线分析 objdump 产生的汇编语言代码完成的，其中 solve_phase_5 和 solve_phase_6 中读取地址制表的操作以及 solve_secret 中 OPEN 作为特殊输出需要出现在 36 字节的限制等少量部分有 AI 协助，但读取汇编语言进行计算的部分均自主完成。